

Security policy

Mine-FNL

Security policy

This document describes the threat model firm-bot is designed to defend against, the controls it ships with, and what is **explicitly out of scope**. It is deliberately honest about limitations — see "Operational notes" for what a real public-internet deployment still needs.

Threat model (STRIDE)

We use the STRIDE framework to enumerate threats. Each category lists firm-bot-specific scenarios the codebase has been hardened against.

S — Spoofing

- **Forged client identity in rate-limit key.** A caller can hide their true IP by hitting firm-bot through proxies that strip or rewrite headers. *Mitigation:* rate limit keys off `scope["client"]` (TCP-level); a future hardened deployment will require `X-Forwarded-For` from a trusted proxy and pin the trusted chain.
- **Forged firm slug via path traversal.** `/v1/firms/{slug}/query` accepts the slug in the URL; without slug validation a caller could request `../../../../etc/passwd`. *Mitigation:* `is_valid_slug` rejects anything outside `[a-z0-9_-]{2,41}` at every endpoint (`firm_bot/config.py:35` and `firm_bot/api/app.py:38`).

T — Tampering

- **Path traversal via upload filename.** `/upload` writes to `source/{filename}`; without sanitisation a caller could write to `../../../../etc/cron.d/....` *Mitigation:* `Path(file.filename).name` strips directories and leading dots (`firm_bot/api/app.py:157`).
- **Configuration tampering via YAML.** The on-disk `config.yaml` is loaded with `yaml.safe_load` only — no Python object deserialisation (`firm_bot/config.py:117`). Env vars override YAML so an operator can correct a misconfigured file without editing it.

R — Repudiation

- **Forged logs leaking secrets.** An operator reads the log file and sees credentials because some code path forgot to scrub them. *Mitigation:* `RedactFilter`

(`firm_bot/security/redact.py`) is applied at logger setup and strips bearer tokens, `api_key=`, `token=`, emails, and any env var ending in `_KEY/_SECRET/_TOKEN`.

- **Missing access log.** We do not yet log every question asked (legacy v0.1 design). *Mitigation:* callers who need a query audit trail should add a reverse proxy with structured access logs.

I — Information disclosure

- **Oversized upload filling memory.** `/upload` streamed without a cap means a 5 GB body OOMs the process. *Mitigation:* `SecurityMiddleware` enforces `max_upload_bytes` (`firm_bot/security/middleware.py:155`); requests exceeding the cap receive 413 *Payload Too Large* before the bytes are read into memory.
- **PII bleed in retrieval.** Documents with SSNs, card numbers, etc. leak into retrieved chunks the LLM re-uses. *Mitigation:* ingest-time `redact_text` (`firm_bot/redact.py`) replaces matches with stable category tokens (`[SSN-REDACTED]`, `[CARD-REDACTED]`, ...) before embedding.
- **Cross-origin data theft.** A browser session on `evil.com` calls `firm-bot` with the user's cookies and reads the response. *Mitigation:* `SecurityMiddleware` defaults to an empty CORS allow list — no `Access-Control-Allow-Origin` header is emitted unless the operator explicitly opts in via `cors_allow_origins`. There is no "default allow" mode.

D — Denial of service

- **Single-client Ollama DoS.** Without rate limiting, one user's `/query/stream` loop saturates Ollama and locks every other tenant out. *Mitigation:* per-client token-bucket rate limit (`firm_bot/security/rate_limit.py`); defaults to 10 req/s sustained, 20 burst per IP.
- **Memory exhaustion from large query bodies.** A caller POSTs a 100 MB question to `/query`. *Mitigation:* body size cap enforced upstream of the route handlers (see "Information disclosure" — same middleware).
- **Bucket-map growth.** A attacker rotating IP source addresses (IPv4 spoofing from a botnet) inflates the in-memory bucket map. *Mitigation:* `RateLimiter` evicts entries idle for more than `t1_seconds` (default 300s).

E — Elevation of privilege

- **No authentication by default.** `firm-bot` ships with an optional API key middleware (`require_api_key: true` in `config.yaml` + `api_keys: [...]`). When enabled, every `/v1/*` endpoint requires `Authorization: Bearer <key>` or `X-API-Key: <key>`. Health, metrics, and the HTML UI are exempt. When disabled, any caller that can reach the HTTP port can act as any operator. *Status:* **opt-in shared-secret gate; per-user auth still requires a reverse proxy** (oauth2-proxy, Pomerium, Cloudflare Access).
- **Multi-tenant cross-read.** A request to `/v1/firms/{slug}/query` for a slug it does not own should still be denied. *Mitigation:* slug is validated; v0.1 has no per-user authorisation layer, so *knowing a slug + an API key* is sufficient. Multi-tenant isolation is a roadmap item, not v0.1.

Controls implemented in this codebase

- **Rate limiting.** Per-IP token bucket, thread-safe, with TTL eviction.
`firm_bot/security/rate_limit.py`, `firm_bot/security/middleware.py`.
- **Request body size cap.** Cumulative-bytes enforcement that returns 413 before reading the entire body into memory. `firm_bot/security/middleware.py:155`.
- **CORS allow-list.** Empty default = no CORS headers; preflight and response-echo paths only emit `Access-Control-*` when the request origin is in `cors_allow_origins`.
`firm_bot/security/middleware.py:198`.
- **API key authentication (opt-in).** Single shared-secret gate applied to `/v1/*` when `require_api_key: true` AND at least one key is configured. Health/metrics/UI exempt. Constant-time key comparison; SHA-256 first-12 used in log lines. Keys are plaintext in config (boundary is "who can read config.yaml"). For real per-user auth, deploy behind oauth2-proxy / Pomerium / Cloudflare Access. `firm_bot/security/api_key.py`.
- **Log redaction.** Filter strips bearer tokens, `api_key=`, `token=`, emails, and `_KEY/_SECRET/_TOKEN` env-var values. `firm_bot/security/redact.py`.
- **Document-level PII redaction.** Ingest-time replacement of SSN, EIN, email, phone, credit card, IBAN, IPv4 with stable category tokens. `firm_bot/redact.py`. Categories are toggleable per firm.
- **Slug validation.** All firm-scoped endpoints reject any slug outside `[a-z0-9][a-z0-9_-]{1,40}`. `firm_bot/config.py:35`.
- **Filename sanitisation on upload.** Strips directory components and leading dots.
`firm_bot/api/app.py:157`.
- **Local-first inference.** No telemetry, no remote calls except to the configured `ollama_host`. The embedding model download is the only outbound network activity on first run; `HF_HUB_OFFLINE=1` disables it.
- **Per-firm data isolation.** Each firm's chunks live in their own Chroma collection and BM25 pickle under `<data_dir>/firms/<slug>/store/`. Slug is the only join key.

Out of scope for v0.1

- **Per-user authentication.** The optional API key middleware is a single shared-secret gate — it doesn't know which user is calling. Per-user auth (login, sessions, OAuth/OIDC) is out of scope; deploy behind an authenticating reverse proxy for that.
- **Per-user authorisation.** All callers with a valid API key can act as any operator on any firm. Tenant isolation is filesystem-level.
- **No encryption at rest.** Chunks and indexes are plain files on disk. If the host is compromised, the documents are readable. Run on an encrypted volume (FileVault, LUKS) for at-rest confidentiality.
- **No CSRF token.** Not currently needed because there is no cookie-based auth and the CORS default is closed. Re-evaluate if cookies are introduced.

Reporting a vulnerability

Email security@firm-bot.example (placeholder). Please do **not** file a public issue for vulnerabilities that affect document confidentiality. For non-security bugs, use the standard GitHub issue template.

Operational notes

- **Single-process assumption.** The token-bucket map is in-memory. A `uvicorn --workers N` deployment has N independent limiters, so the effective per-IP budget is $N \times \text{rate}$. For shared-budget deployments, swap `TokenBucket` for a Redis-backed implementation (the `RateLimiter.check` signature is the only thing to keep stable).
- **Trusted proxy chain.** `client_key_from_scope` uses `scope["client"]`. When terminating at a reverse proxy, you must either (a) keep the limiter on the trusted proxy and rely on its rate-limit feature, or (b) modify `client_key_from_scope` to honour `X-Forwarded-For` *only when* the connection peer is on an allow list. Honouring `X-Forwarded-For` unconditionally lets callers forge their bucket key.
- **Size limits interact.** `max_upload_bytes` (`firm_bot/config.py`, default 200 MB) is the *hard* cap used by `/upload` business logic. `body_max_bytes` (default 1 MB) is enforced by the middleware and applies to *every* request, not just uploads. Tune `body_max_bytes` high enough to accommodate the largest query / eval body you expect — `/upload` already uses its own higher cap.
- **CORS.** Set `cors_allow_origins` to a comma-separated list of origins (`https://app.example.com,https://admin.example.com`), or to `*` for development only. The empty default is fail-closed.